計算機科学実験及演習 3 ハードウェア

SIMPLE/B プロセッサ設計レポート

基本アーキテクチャの実装と検証

題目	SIMPLE/B プロセッサの設計と実装
学籍番号	（学籍番号を記入）
氏名	（氏名を記入）
提出日	2026 年 5 月 8 日

京都大学 工学部情報学科 計算機科学コース

目次

1 **はじめに** 2

2 **実験環境** 2

3 **SIMPLE アーキテクチャ概要** 2
- 3.1 レジスタとメモリ 2
- 3.2 命令セットアーキテクチャ 2

4 **SIMPLE/B マイクロアーキテクチャ** 3
- 4.1 全体構成 3
- 4.2 フェーズの動作 4
- 4.3 データパスの選択 4
- 4.4 制御信号 5

5 **モジュール設計** 6
- 5.1 モジュール構成 6
- 5.2 ALU (`alu.v`) 6
- 5.3 シフタ (`shifter.v`) 8
- 5.4 レジスタファイル (`regfile.v`) 9
- 5.5 主記憶 (`mem.v`) 9
- 5.6 CPU コア (`simple_cpu.v`) 10
- 5.7 トップモジュール (`simple_top.v`) 17

6 **テストプログラム** 21
- 6.1 基本テスト: 加算と出力 21
- 6.2 応用テスト: 遅延付きカウンタ 21

7 **動作検証** 22
- 7.1 テストベンチ 22
- 7.2 シミュレーション結果 25
- 7.3 命令実行トレース 25

8 **性能評価** 26
- 8.1 命令実行サイクル数 26
- 8.2 回路規模の見積もり 26
- 8.3 最大動作周波数 26
- 8.4 改良の方向性 27

9 **まとめ** 27

1 はじめに

本レポートでは、SIMPLE (SIxteen-bit MicroProcessor for Laboratory Experiment) アーキテクチャに準拠したプロセッサ SIMPLE/B の設計と実装について報告する。SIMPLE/B は設計資料（ver 4.0）に記載された基本設計に基づく 5 フェーズ順次実行方式のプロセッサであり、Verilog HDL で記述し、Icarus Verilog によるシミュレーションで動作を検証した。

2 実験環境

表 1　実験環境

項目	内容
FPGA ボード	PowerMedusa MU500-RX/RK
FPGA	Altera Cyclone IV EP4CE30F23I7N
CAD ツール	Intel Quartus Prime 20.1 Standard Edition
シミュレータ	Icarus Verilog 11.0 / ModelSim-Intel FPGA Starter Edition
OS	Ubuntu 22.04 LTS (WSL2)
クロック	20MHz 発振器

3 SIMPLE アーキテクチャ概要

3.1 レジスタとメモリ

SIMPLE は 1 語 16 ビットのコンピュータであり、以下の資源を持つ。

- **主記憶**: 16 ビットアドレス、64KW（語単位）のアドレス空間。Cyclone IV EP4CE30 では RAM 容量の制限により最大 33KW。本実装では 256 語（8 ビットアドレス）の分散 RAM を使用。
- **汎用レジスタ**: 8 本（`r[0]`〜`r[7]`）、各 16 ビット。
- **プログラムカウンタ (PC)**: 実行中の命令のアドレスを保持。
- **条件コード**: S（符号）、Z（ゼロ）、C（桁上げ）、V（オーバーフロー）の 4 ビット。

3.2 命令セットアーキテクチャ

SIMPLE の命令は全て 1 語（16 ビット）の固定長であり、4 種類の命令形式を持つ。

3.2.1 演算/入出力命令

3.2.2 ロード/ストア命令

- **LD Ra, d(Rb)**: `r[Ra] = *(r[Rb] + sign_ext(d))` — レジスタ間接 + 変位

表 2　命令形式

形式	ビット配置	用途
(a) 演算/IO	`11|Rs(3)|Rd(3)|op3(4)|d(4)`	ALU, シフト, IN/OUT, HLT
(b) LD/ST	`op1(2)|Ra(3)|Rb(3)|d(8)`	メモリアクセス
(c) LI/B	`10|op2(3)|Rb(3)|d(8)`	即値ロード, 無条件分岐
(d) Bcc	`10|111|cond(3)|d(8)`	条件分岐

表 3　演算/入出力命令一覧

ニーモニック	op3	機能	フラグ設定
ADD Rd,Rs	0000	`r[Rd] = r[Rd] + r[Rs]`	S,Z,C,V
SUB Rd,Rs	0001	`r[Rd] = r[Rd] - r[Rs]`	S,Z,C,V
AND Rd,Rs	0010	`r[Rd] = r[Rd] & r[Rs]`	S,Z; C=0
OR Rd,Rs	0011	`r[Rd] = r[Rd] | r[Rs]`	S,Z; C=0
XOR Rd,Rs	0100	`r[Rd] = r[Rd] ^ r[Rs]`	S,Z; C=0
CMP Rd,Rs	0101	`r[Rd] - r[Rs]`（フラグのみ）	S,Z,C,V
MOV Rd,Rs	0110	`r[Rd] = r[Rs]`	S,Z; C=0
SLL Rd,d	1000	左論理シフト	S,Z,C; V=0
SLR Rd,d	1001	左循環シフト	S,Z; C=0, V=0
SRL Rd,d	1010	右論理シフト	S,Z,C; V=0
SRA Rd,d	1011	右算術シフト	S,Z,C; V=0
IN Rd	1100	`r[Rd] = input`	なし
OUT Rs	1101	`output = r[Rs]`	なし
HLT	1111	停止	なし

- **ST Ra, d(Rb)**: `*(r[Rb] + sign_ext(d)) = r[Ra]`

3.2.3　即値ロード/分岐命令

- **LI Rb, d**: `r[Rb] = sign_ext(d)` — 8 ビット即値の符号拡張
- **B d**: `PC = PC + 1 + sign_ext(d)` — PC 相対無条件分岐

3.2.4　条件分岐命令

4　SIMPLE/B マイクロアーキテクチャ

4.1　全体構成

SIMPLE/B は 1 命令を 5 つのフェーズに分割し、順次活性化しながら実行する。図 1 にブロック図、図 2 にフェーズのタイミングを示す。

表 4　条件分岐命令

ニーモニック	cond	分岐条件
BE d	000	$Z = 1$（等しい）
BLT d	001	$S \oplus V = 1$（符号付き小なり）
BLE d	010	$Z \vee (S \oplus V) = 1$（符号付き以下）
BNE d	011	$Z = 0$（等しくない）

4.2　フェーズの動作

1. **p1 — 命令フェッチ (IF)**: PC が保持するアドレスの命令を主記憶からフェッチし、IR (Instruction Register) に格納する。同時に PC を 1 加算する。
2. **p2 — レジスタ読み出し (RR)**: IR の `Rs/Ra` フィールド（`I[13:11]`）で指定されるレジスタの値を AR に、`Rd/Rb` フィールド（`I[10:8]`）で指定されるレジスタの値を BR に格納する。
3. **p3 — 演算 (EX)**: 命令に応じて ALU またはシフタで演算を行い、結果を DR (Data Register) に格納する。演算命令では条件コード S, Z, C, V を更新する。LD/ST 命令では BR + sign_ext(d) により実効アドレスを計算する。分岐命令では PC + sign_ext(d) により分岐先アドレスを計算する。
4. **p4 — 主記憶アクセス / 入出力 (MA)**: LD 命令では DR の値をアドレスとして主記憶を読み出し、MDR に格納する。ST 命令では AR の値（`r[Ra]`）を DR のアドレスに書き込む。IN 命令では外部入力を MDR に格納し、OUT 命令では AR の値を外部出力する。
5. **p5 — レジスタ書き込み (WB)**: DR または MDR の値を、命令で指定されたレジスタに書き込む。分岐命令では DR の値を PC に書き込む。HLT 命令では実行を停止する。

4.3　データパスの選択

各フェーズにおけるデータパスの選択を表 5 に示す。

表 5　命令種別ごとのデータパス

命令	p2		p3			p4	p5	
	AR	BR	ALU 入力 A	ALU 入力 B	結果	動作	書込先	データ
ALU	r[Rs]	r[Rd]	BR	AR	DR	—	r[Rd]	DR
CMP	r[Rs]	r[Rd]	BR	AR	—	—	—	—
Shift	—	r[Rd]	—	—	DR	—	r[Rd]	DR
LD	r[Ra]	r[Rb]	BR	sext(d)	DR	MDR←mem[DR]	r[Ra]	MDR
ST	r[Ra]	r[Rb]	BR	sext(d)	DR	mem[DR]←AR	—	—
LI	—	—	0	sext(d)	DR	—	r[Rb]	DR
B	—	—	PC	sext(d)	DR	—	PC	DR
Bcc	—	—	PC	sext(d)	DR	—	PC/—	DR
IN	—	—	—	—	—	MDR←input	r[Rd]	MDR
OUT	r[Rs]	—	—	—	—	output←AR	—	—

```
        +----------+
+------>|    PC     |------+----> アドレスバス
|       +----------+      |
|           |  +1         v
|           v          +------+
|  p1  +----------+     | 主記憶 |
+------|   IR     |<---|      |<--- データバス
|      +----------+     +------+
|         デコード          ^
|           |  |          |
|      p2   v  v          |
|      +----+----+        |
|      | AR | BR |        |
|      +----+----+        |
|        |    |           |
|   p3   v    v           |
|      +--------+  +-------+
|      |  ALU   |  |シフタ  |
|      +--------+  +-------+
|         |  |          |
|    p4   v  v          |
|      +------+ +-----+
|      | SZCV | | DR  |----+----> アドレスバス
|      +------+ +-----+    |
|                  |       |
|   p5          +------+   |
+---------------|  MDR |<--+<--- 外部入力
                +------+
                   |-----> 外部出力
```

図 1　SIMPLE/B ブロック図（簡略版）

4.4　制御信号

制御部はフェーズカウンタ（3 ビット、0〜4 を巡回）と命令デコード結果に基づき、以下の制御を行う。

- **メモリアドレス選択**: p1 では PC、p4 では DR
- **メモリ書き込み許可**: p4 かつ ST 命令のときのみ有効
- **レジスタ書き込み**: p5 かつ書き込みを伴う命令（ALU/Shift/LD/LI/IN）のとき有効。書き

```
clock  __/~~\__/~~\__/~~\__/~~\__/~~\__/~~\__
phase   | p1 | p2 | p3 | p4 | p5 | p1 | ...
        | IF | RR | EX | MA | WB | IF | ...
```

図 2　SIMPLE/B の 5 フェーズ実行

込み先は LD 命令では `I[13:11]`（Ra）、その他では `I[10:8]`（Rd/Rb）。

- **exec 制御**: exec の立ち上がりエッジで実行開始、HLT 命令で停止

5　モジュール設計

5.1　モジュール構成

表 6　モジュール一覧

ファイル	モジュール名	機能
`alu.v`	`alu`	算術論理演算ユニット。ADD/SUB/AND/OR/XOR/CMP/MOV の 7 演算と条件コード生成。
`shifter.v`	`shifter`	バレルシフタ。SLL/SLR/SRL/SRA の 4 種シフトと条件コード C 生成。
`regfile.v`	`regfile`	8×16 ビットレジスタファイル。非同期 2 リード、同期 1 ライト。
`simple_cpu.v`	`simple_cpu`	CPU コア。5 フェーズ FSM、命令デコード、データパス制御。
`mem.v`	`mem`	主記憶。非同期リード、同期ライト。`$readmemh` で初期化。
`simple_top.v`	`simple_top`	トップモジュール。CPU+ メモリ +7 セグ表示 +I/O 接続。
`seg7dec.v`	`seg7dec`	7 セグメントデコーダ（導入課題から流用）。

5.2　ALU (`alu.v`)

16 ビット 2 入力の ALU で、加算・減算・論理演算・比較・転送の 7 演算を行う。減算は 2 の補数加算（`a + ~b + 1`）で実装し、CMP は SUB と同じ演算でフラグのみ設定する。

```
// alu.v - SIMPLE/B ALU
// ADD, SUB, AND, OR, XOR, CMP, MOV + アドレス計算(ADD流用)

module alu(
    input  [15:0] a,
    input  [15:0] b,
    input  [3:0]  op,
```

```
    output [15:0] result,
    output        flag_s,
    output        flag_z,
    output        flag_c,
    output        flag_v
);

wire [16:0] sum  = {1'b0, a} + {1'b0, b};
wire [16:0] diff = {1'b0, a} + {1'b0, ~b} + 17'd1;

assign result = (op == 4'h0) ? sum[15:0]  :  // ADD
                (op == 4'h1) ? diff[15:0]  :  // SUB
                (op == 4'h2) ? (a & b)     :  // AND
                (op == 4'h3) ? (a | b)     :  // OR
                (op == 4'h4) ? (a ^ b)     :  // XOR
                (op == 4'h5) ? diff[15:0]  :  // CMP (= SUB)
                (op == 4'h6) ? b           :  // MOV
                               16'h0000;

assign flag_s = result[15];
assign flag_z = (result == 16'h0000);

assign flag_c = (op == 4'h0)                ? sum[16]  :
                (op == 4'h1 || op == 4'h5)  ? diff[16] :
                                              1'b0;

wire add_ov = (~a[15] & ~b[15] & result[15]) | (a[15] & b[15] & ~result[15]);
wire sub_ov = (~a[15] &  b[15] & result[15]) | (a[15] & ~b[15] & ~result[15]);

assign flag_v = (op == 4'h0)                ? add_ov :
                (op == 4'h1 || op == 4'h5)  ? sub_ov :
                                              1'b0;

endmodule
```

Listing 1　alu.v

5.2.1　条件コード生成

- **S**: 演算結果の最上位ビット（`result[15]`）
- **Z**: 演算結果が 0 なら 1
- **C**: ADD では 17 ビット加算の桁上げ、SUB/CMP では 2 の補数加算の桁上げ（$a \geq b$ のとき 1）。論理演算・MOV では 0。
- **V**: ADD/SUB/CMP のみ。符号付きオーバーフロー検出。**（同符号の入力）** ∧ **（結果の符号が異なる）** で判定。

5.3 シフタ (shifter.v)

16 ビットデータを 0〜15 ビットシフトするバレルシフタ。4 種のシフト演算に対応する。

```
// shifter.v - SIMPLE/B バレルシフタ
// SLL(00), SLR(01), SRL(10), SRA(11)

module shifter(
    input  [15:0] data,
    input  [3:0]  d,
    input  [1:0]  op,
    output reg [15:0] result,
    output reg        flag_c
);

wire [16:0] sll_ext = {1'b0, data} << d;
wire [16:0] srl_ext = {data, 1'b0} >> d;

always @(*) begin
    case (op)
        2'b00: begin // SLL
            result = sll_ext[15:0];
            flag_c = (d == 4'd0) ? 1'b0 : sll_ext[16];
        end
        2'b01: begin // SLR (左循環シフト)
            result = (data << d) | (data >> (5'd16 - {1'b0, d}));
            flag_c = 1'b0;
        end
        2'b10: begin // SRL
            result = data >> d;
            flag_c = (d == 4'd0) ? 1'b0 : srl_ext[0];
        end
        2'b11: begin // SRA
            result = $signed(data) >>> d;
            flag_c = (d == 4'd0) ? 1'b0 : srl_ext[0];
        end
    endcase
end

endmodule
```

Listing 2　shifter.v

条件コード C は、シフト桁数が 0 または SLR（循環シフト）の場合は 0、それ以外では最後にシフトアウトされたビットの値が設定される。V は常に 0。

5.4 レジスタファイル (regfile.v)

8 本の 16 ビット汎用レジスタを提供する。2 つの読み出しポート（非同期、組み合わせ回路）と 1 つの書き込みポート（同期）を持つ。

```
// regfile.v - SIMPLE/B レジスタファイル (8 x 16bit)
// 2リード / 1ライトポート, 非同期リード

module regfile(
    input              clk,
    input              rst_n,
    input      [2:0]   rd_addr0,
    input      [2:0]   rd_addr1,
    output     [15:0]  rd_data0,
    output     [15:0]  rd_data1,
    input              we,
    input      [2:0]   wr_addr,
    input      [15:0]  wr_data
);

reg [15:0] regs [0:7];

assign rd_data0 = regs[rd_addr0];
assign rd_data1 = regs[rd_addr1];

integer i;
always @(posedge clk or negedge rst_n) begin
    if (~rst_n) begin
        for (i = 0; i < 8; i = i + 1)
            regs[i] <= 16'h0000;
    end else if (we) begin
        regs[wr_addr] <= wr_data;
    end
end

endmodule
```

Listing 3　regfile.v

非同期リードにより、p2 でレジスタ値が即座に AR/BR に供給される。書き込みは p5 で `rf_we` が有効な間にクロックエッジで実行される。

5.5 主記憶 (mem.v)

```
// mem.v - SIMPLE/B 主記憶
// 非同期リード / 同期ライト
// $readmemh でプログラムを初期化

```

```
module mem(
    input          clk,
    input  [15:0] addr,
    input  [15:0] wdata,
    input          we,
    output [15:0] rdata
);

parameter ADDR_WIDTH = 8;  // 256語（デフォルト）
parameter INIT_FILE  = "test.hex";

reg [15:0] ram [0:(1 << ADDR_WIDTH) - 1];

initial begin
    $readmemh(INIT_FILE, ram);
end

assign rdata = ram[addr[ADDR_WIDTH-1:0]];

always @(posedge clk) begin
    if (we)
        ram[addr[ADDR_WIDTH-1:0]] <= wdata;
end

endmodule
```

Listing 4　mem.v

非同期リード（assign 文）と同期ライト（always @(posedge clk)）を持つ。FPGA 上では Distributed RAM として実装される。$readmemh によりプログラムを初期化する。アドレス幅はパラメータで変更可能（デフォルト 8 ビット = 256 語）。

5.6　CPU コア (simple_cpu.v)

SIMPLE/B の全制御を行うモジュール。フェーズカウンタによる FSM とデータパス制御を含む。

```
// simple_cpu.v - SIMPLE プロセッサ（拡張版）
//
// ■ マイクロアーキテクチャ: 4フェーズ実行（SIMPLE/Bの5フェーズから高速化）
//   PH1 (IF+WB): 命令フェッチ + 前命令のレジスタ書き戻し
//   PH2 (RR):    レジスタ読み出し
//   PH3 (EX):    演算 + ST書込み
//   PH4 (MA):    メモリ読出し / IO / 分岐 / WB準備
//
// ■ ISA拡張（基本アーキテクチャに追加）:
//   BAL Rb, d  (op2=001): r[Rb] = PC, PC = PC + sign_ext(d)  -- Branch And Link
//   BR  Rb     (op2=010): PC = r[Rb]                         -- Branch Register
```

```
//   ADDI Rb, d (op2=011): r[Rb] = r[Rb] + sign_ext(d)        -- Add Immediate
//
// ■ 同期RAM対応: アドレスを1フェーズ前に提示
//   PH3: LD/ST用実効アドレス → PH4でデータ取得 / PH3→PH4でST書込み
//   PH4: 次命令アドレス      → PH1で命令取得

module simple_cpu(
    input         clk,
    input         rst_n,
    input         exec,
    // メモリインタフェース
    output [15:0] mem_addr,
    input  [15:0] mem_rdata,
    output [15:0] mem_wdata,
    output        mem_we,
    // I/O インタフェース
    input  [15:0] in_data,
    output reg [15:0] out_data,
    output reg        out_we,
    output [3:0]  io_port,    // I/Oポート番号 (d4)
    output        io_read,    // IN命令実行中 (PH4)
    output        io_write,   // OUT命令実行中 (PH4)
    // デバッグポート
    output [15:0] debug_pc,   // 現在のPC値
    output [15:0] debug_r0,   // レジスタr0値
    output [15:0] debug_r1,   // レジスタr1値
    output [15:0] debug_r2,   // レジスタr2値
    output [15:0] debug_r3,   // レジスタr3値
    // ステータス
    output        halted
);

// ---- フェーズ定義 (4サイクル) ----
localparam PH1 = 2'd0; // IF + WB
localparam PH2 = 2'd1; // RR
localparam PH3 = 2'd2; // EX + ST write
localparam PH4 = 2'd3; // MA / IO / Branch / WB setup

// ---- 内部レジスタ ----
reg [15:0] pc, ir, ar, br, dr, mdr;
reg [1:0]  phase;
reg        flag_s, flag_z, flag_c, flag_v;
reg        running;

// ---- 書き戻しパイプラインレジスタ ----
reg        wb_rf_we;
reg [2:0]  wb_rf_addr;
reg [15:0] wb_rf_data;  // ALU/Shift/LI/ADDI/BAL 用
```

```
reg         wb_use_mdr;  // LD/IN: MDR を使用

// ---- 命令デコード ----
wire [1:0] op1   = ir[15:14];
wire [2:0] rs_ra = ir[13:11];
wire [2:0] rd_rb = ir[10:8];
wire [3:0] op3   = ir[7:4];
wire [3:0] d4    = ir[3:0];
wire [7:0] d8    = ir[7:0];
wire [2:0] op2   = ir[13:11];
wire [2:0] cond  = ir[10:8];

wire [15:0] sext_d8 = {{8{d8[7]}}, d8};

// 命令種別
wire is_alu   = (op1 == 2'b11);
wire is_load  = (op1 == 2'b00);
wire is_store = (op1 == 2'b01);
wire is_li    = (op1 == 2'b10) && (op2 == 3'b000);
wire is_bal   = (op1 == 2'b10) && (op2 == 3'b001); // 拡張
wire is_br    = (op1 == 2'b10) && (op2 == 3'b010); // 拡張
wire is_addi  = (op1 == 2'b10) && (op2 == 3'b011); // 拡張
wire is_b     = (op1 == 2'b10) && (op2 == 3'b100);
wire is_bcc   = (op1 == 2'b10) && (op2 == 3'b111);

wire is_shift  = is_alu && op3[3] && ~op3[2];       // 1000-1011
wire is_in     = is_alu && (op3 == 4'b1100);
wire is_out    = is_alu && (op3 == 4'b1101);
wire is_hlt    = is_alu && (op3 == 4'b1111);
wire is_alu_rr = is_alu && ~op3[3] && (op3 != 4'b0111); // 0000-0110

// 分岐判定
reg branch_taken;
always @(*) begin
    case (cond)
        3'b000:  branch_taken = flag_z;
        3'b001:  branch_taken = flag_s ^ flag_v;
        3'b010:  branch_taken = flag_z | (flag_s ^ flag_v);
        3'b011:  branch_taken = ~flag_z;
        default: branch_taken = 1'b0;
    endcase
end

wire branch_go = is_b || is_bal || is_br || (is_bcc && branch_taken);

// レジスタ書き込みが必要な命令
wire need_rf_write = (is_alu_rr && op3 != 4'b0101) || // ALU (CMP除く)
                     is_shift || is_load || is_li ||
                     is_in || is_bal || is_addi;
```

```
109
110 // ---- レジスタファイル ----
111 wire [15:0] rf_rd0, rf_rd1;
112 wire        rf_we      = (phase == PH1) && running && wb_rf_we;
113 wire [2:0]  rf_wr_addr = wb_rf_addr;
114 wire [15:0] rf_wr_data = wb_use_mdr ? mdr : wb_rf_data;
115
116 regfile rf(
117     .clk(clk), .rst_n(rst_n),
118     .rd_addr0(rs_ra), .rd_data0(rf_rd0),
119     .rd_addr1(rd_rb), .rd_data1(rf_rd1),
120     .we(rf_we), .wr_addr(rf_wr_addr), .wr_data(rf_wr_data)
121 );
122
123 // ---- ALU ----
124 wire [15:0] alu_a = is_li            ? 16'h0000 :
125                     (is_b || is_bcc || is_bal) ? pc :
126                                                  br;
127 wire [15:0] alu_b = is_alu ? ar : sext_d8;
128 wire [3:0]  alu_op = is_alu ? op3 : 4'h0; // 非ALU命令はADD
129
130 wire [15:0] alu_result;
131 wire        alu_s, alu_z, alu_c, alu_v;
132
133 alu alu_inst(
134     .a(alu_a), .b(alu_b), .op(alu_op),
135     .result(alu_result),
136     .flag_s(alu_s), .flag_z(alu_z), .flag_c(alu_c), .flag_v(alu_v)
137 );
138
139 // ---- シフタ ----
140 wire [15:0] shift_result;
141 wire        shift_c;
142
143 shifter shift_inst(
144     .data(br), .d(d4), .op(op3[1:0]),
145     .result(shift_result), .flag_c(shift_c)
146 );
147
148 // ---- メモリインタフェース ----
149 // 同期RAM: PH3でアドレス提示 → PH4でデータ取得 / PH3→PH4でST書込み
150 //          PH4でnext_pc提示 → PH1で命令取得
151 wire [15:0] next_pc_val = branch_go ? dr : pc;
152
153 assign mem_addr  = (phase == PH3 && (is_load || is_store)) ? alu_result :
154                   (phase == PH4)                            ? next_pc_val :
155                                                               pc;
156 assign mem_wdata = ar;
157 assign mem_we    = (phase == PH3) && running && is_store;
```

```
158
159 // ---- I/Oポート制御 ----
160 assign io_port  = d4;
161 assign io_read  = (phase == PH4) && running && is_in;
162 assign io_write = (phase == PH4) && running && is_out;
163
164 // ---- デバッグポート ----
165 assign debug_pc = pc;
166 assign debug_r0 = rf.regs[0];
167 assign debug_r1 = rf.regs[1];
168 assign debug_r2 = rf.regs[2];
169 assign debug_r3 = rf.regs[3];
170
171 // ---- ステータス ----
172 assign halted = ~running;
173
174 // ---- exec 立ち上がり検出 ----
175 reg exec_prev;
176 always @(posedge clk or negedge rst_n) begin
177     if (~rst_n) exec_prev <= 1'b0;
178     else        exec_prev <= exec;
179 end
180 wire exec_rise = exec & ~exec_prev;
181
182 // ======== メインFSM ========
183 always @(posedge clk or negedge rst_n) begin
184     if (~rst_n) begin
185         pc          <= 16'h0000;
186         ir          <= 16'h0000;
187         ar          <= 16'h0000;
188         br          <= 16'h0000;
189         dr          <= 16'h0000;
190         mdr         <= 16'h0000;
191         phase       <= PH1;
192         flag_s      <= 1'b0;
193         flag_z      <= 1'b0;
194         flag_c      <= 1'b0;
195         flag_v      <= 1'b0;
196         running     <= 1'b0;
197         out_data    <= 16'h0000;
198         out_we      <= 1'b0;
199         wb_rf_we    <= 1'b0;
200         wb_rf_addr  <= 3'd0;
201         wb_rf_data  <= 16'h0000;
202         wb_use_mdr  <= 1'b0;
203     end else begin
204         out_we <= 1'b0;
205
206         if (~running && exec_rise)
```

```
            running <= 1'b1;

        if (running) begin
            case (phase)
                // ============ PH1: 命令フェッチ + 書き戻し ============
                // IR ← mem[next_pc] (PH4で提示済み)
                // WB: rf_we は組み合わせ信号、このエッジでレジスタに書込み
                PH1: begin
                    ir    <= mem_rdata;
                    pc    <= pc + 16'd1;
                    phase <= PH2;
                end

                // ============ PH2: レジスタ読み出し ============
                PH2: begin
                    ar    <= rf_rd0; // r[Rs/Ra]
                    br    <= rf_rd1; // r[Rd/Rb]
                    phase <= PH3;
                end

                // ============ PH3: 演算 + ST書込み ============
                PH3: begin
                    // DR ← 演算結果
                    if (is_shift)
                        dr <= shift_result;
                    else
                        dr <= alu_result;

                    // 条件コード更新
                    if (is_alu_rr || is_addi) begin
                        flag_s <= alu_s;
                        flag_z <= alu_z;
                        flag_c <= alu_c;
                        flag_v <= alu_v;
                    end else if (is_shift) begin
                        flag_s <= shift_result[15];
                        flag_z <= (shift_result == 16'h0000);
                        flag_c <= shift_c;
                        flag_v <= 1'b0;
                    end

                    // ST: mem_we=1 (組み合わせ)、PH3→PH4エッジで書込み
                    phase <= PH4;
                end

                // ============ PH4: メモリ読出し / IO / 分岐 / WB準備
                    ============
                PH4: begin
                    // LD: MDR ← mem[DR] (PH3でアドレス提示済み)
```

```
                    if (is_load)
                        mdr <= mem_rdata;

                    // IN / OUT
                    if (is_in)
                        mdr <= in_data;
                    if (is_out) begin
                        out_data <= ar;
                        out_we   <= 1'b1;
                    end

                    // 分岐: PC更新
                    if (branch_go)
                        pc <= dr;

                    // HLT
                    if (is_hlt)
                        running <= 1'b0;

                    // 書き戻し情報をパイプラインレジスタに保存
                    wb_rf_we   <= need_rf_write;
                    wb_rf_addr <= is_load ? rs_ra : rd_rb;
                    wb_use_mdr <= (is_load || is_in);
                    if (is_bal)
                        wb_rf_data <= pc; // 戻りアドレス (= 命令アドレス +
                            1)
                    else
                        wb_rf_data <= dr;

                    // PH4→PH1エッジ: RAMが next_pc_val をキャプチャ
                    phase <= PH1;
                end
            endcase
        end
    end
end

endmodule
```

Listing 5　simple_cpu.v

5.6.1　命令デコード

IR の上位 2 ビット（op1）で命令形式を判別し、演算命令では op3 フィールドで具体的な演算を特定する。デコード結果は組み合わせ回路として生成され、各フェーズの制御に使用される。

5.6.2　ALU 入力マルチプレクサ

ALU 入力 A は命令種別に応じて以下から選択される:

- ALU 演算・LD/ST: BR（`r[Rd]` または `r[Rb]`）
- LI: 定数 0（即値をそのまま通すため）
- B/Bcc: PC（分岐先アドレス計算のため）

ALU 入力 B は:

- ALU 演算: AR（`r[Rs]`）
- LD/ST/LI/B/Bcc: `sign_ext(d8)`（8 ビット即値/変位の符号拡張）

5.6.3 レジスタ書き込み制御

書き込みイネーブル信号 `rf_we` は組み合わせ論理として生成される。これにより、p5 のフェーズ中に `rf_we = 1` となり、p5→p1 のクロックエッジでレジスタファイルへの書き込みが実行される。

5.7 トップモジュール (`simple_top.v`)

CPU、メモリ、7 セグメント表示を統合するトップモジュール。OUT 命令で出力された 16 ビット値をラッチし、ダイナミック点灯により 4 桁の 16 進数として 7 セグメント LED に表示する。

```
// simple_top.v - SIMPLE/B トップモジュール（担当B拡張版）
// CPU + メモリ + タイマ + I/Oアドレスデコード + 8桁7セグ表示
//
// I/Oポートマップ:
//   ポート0: IN=DIPスイッチA(SW26), OUT=7セグ出力(下位16bit)
//   ポート1: IN=DIPスイッチB(SW27), OUT=7セグ出力(上位16bit)
//   ポート2: IN=タイマCOUNT,        OUT=タイマCTRL
//   ポート3: IN=タイマoverflow,      OUT=タイマLIMIT
//   ポート4: IN=プッシュスイッチ,    OUT=LED出力(8bit)
//   ポート5: IN=ロータリースイッチ,  OUT=ブザー制御
//
// 7セグ表示モード（ロータリースイッチで切替):
//   モード0: OUTレジスタ値（デフォルト）
//   モード1: PC値（デバッグ用）
//   モード2: レジスタr0~r3値
//   モード3: タイマCOUNT値

module simple_top(
    input          clk,
    input          rst_n,
    input          exec,
    input  [15:0]  sw,        // DIPスイッチ A+B（負論理）
    input  [3:0]   pushsw,    // プッシュスイッチ（負論理）
    input  [3:0]   rotsw,     // ロータリースイッチ
    output [7:0]   seg,       // 7セグメントLEDセグメント出力
    output [7:0]   sel,       // 7セグメントLED桁選択（8桁）
    output reg [7:0] led      // LED出力
);

```

```
// ---- CPU ↔ メモリ接続 ----
wire [15:0] mem_addr, mem_rdata, mem_wdata;
wire        mem_we;

// ---- CPU ↔ I/O ----
wire [15:0] cpu_out_data;
wire        cpu_out_we;
wire [3:0]  io_port;
wire        io_read, io_write;
wire        cpu_halted;

// ---- デバッグ信号 ----
wire [15:0] debug_pc, debug_r0, debug_r1, debug_r2, debug_r3;

// ---- I/O 入力マルチプレクサ ----
reg [15:0] in_mux;

simple_cpu cpu(
    .clk(clk), .rst_n(rst_n), .exec(exec),
    .mem_addr(mem_addr), .mem_rdata(mem_rdata),
    .mem_wdata(mem_wdata), .mem_we(mem_we),
    .in_data(in_mux),
    .out_data(cpu_out_data), .out_we(cpu_out_we),
    .io_port(io_port), .io_read(io_read), .io_write(io_write),
    .debug_pc(debug_pc),
    .debug_r0(debug_r0), .debug_r1(debug_r1),
    .debug_r2(debug_r2), .debug_r3(debug_r3),
    .halted(cpu_halted)
);

// ---- メモリ (Block RAM) ----
ram memory(
    .clock(clk),
    .address(mem_addr[11:0]),
    .q(mem_rdata),
    .data(mem_wdata),
    .wren(mem_we)
);

// ---- タイマモジュール ----
wire [15:0] timer_count;
wire        timer_overflow;

timer timer_inst(
    .clk(clk), .rst_n(rst_n),
    .wdata(cpu_out_data),
    .we_ctrl(io_write && (io_port == 4'd2)),
    .we_limit(io_write && (io_port == 4'd3)),
    .rd_overflow(io_read && (io_port == 4'd3)),
```

```
    .count(timer_count),
    .overflow(timer_overflow)
);

// ---- I/O 入力デコーダ ----
always @(*) begin
    case (io_port)
        4'd0:    in_mux = {8'h00, ~sw[7:0]};        // DIPスイッチ A（負論理反転）
        4'd1:    in_mux = {8'h00, ~sw[15:8]};       // DIPスイッチ B（負論理反転）
        4'd2:    in_mux = timer_count;                // タイマ COUNT
        4'd3:    in_mux = {15'd0, timer_overflow};    // タイマ overflow
        4'd4:    in_mux = {12'h000, ~pushsw};         // プッシュスイッチ（負論理反転）
        4'd5:    in_mux = {12'h000, rotsw};           // ロータリースイッチ
        default: in_mux = 16'h0000;
    endcase
end

// ---- I/O 出力レジスタ ----
reg [15:0] seg_reg_lo;   // 7セグ下位16bit（ポート0）
reg [15:0] seg_reg_hi;   // 7セグ上位16bit（ポート1）

always @(posedge clk or negedge rst_n) begin
    if (~rst_n) begin
        seg_reg_lo <= 16'h0000;
        seg_reg_hi <= 16'h0000;
        led        <= 8'h00;
    end else if (io_write) begin
        case (io_port)
            4'd0: seg_reg_lo <= cpu_out_data;
            4'd1: seg_reg_hi <= cpu_out_data;
            4'd4: led        <= cpu_out_data[7:0];
        endcase
    end
end

// 後方互換: 従来の OUT（ポート番号なし）も seg_reg_lo に書き込み
// cpu_out_we は全OUT命令で立つので、ポート0以外の場合もフォールバック
// → io_write でポート区別するのでこの処理は不要

// ---- 7セグ表示（8桁対応）----
parameter DISP_DIV = 5000; // 20MHz / 5000 = 4kHz

reg [12:0] disp_cnt;
wire       disp_en = (disp_cnt == DISP_DIV - 1);

always @(posedge clk or negedge rst_n) begin
```

```
    if (~rst_n)        disp_cnt <= 13'd0;
    else if (disp_en) disp_cnt <= 13'd0;
    else               disp_cnt <= disp_cnt + 13'd1;
end

reg [2:0] dig_sel;  // 8桁: 0~7
always @(posedge clk or negedge rst_n) begin
    if (~rst_n)        dig_sel <= 3'd0;
    else if (disp_en) dig_sel <= dig_sel + 3'd1;
end

// 表示モード選択（ロータリースイッチ下位2ビット）
wire [1:0] disp_mode = rotsw[1:0];

// 表示データ選択 (32bit)
// モード2: r0~r3 の下位8ビットを並べて8桁表示
//   桁7(左端)=r3[7:4], 桁6=r3[3:0], ... 桁1=r0[7:4], 桁0(右端)=r0[3:0]
reg [31:0] disp_data;
always @(*) begin
    case (disp_mode)
        2'd0: disp_data = {seg_reg_hi, seg_reg_lo};
        2'd1: disp_data = {16'h0000, debug_pc};
        2'd2: disp_data = {debug_r3[7:0], debug_r2[7:0],
                           debug_r1[7:0], debug_r0[7:0]};
        2'd3: disp_data = {16'h0000, timer_count};
    endcase
end

// 現在の桁のニブル (4bit)
reg [3:0] cur_dig;
always @(*) begin
    case (dig_sel)
        3'd0: cur_dig = disp_data[3:0];
        3'd1: cur_dig = disp_data[7:4];
        3'd2: cur_dig = disp_data[11:8];
        3'd3: cur_dig = disp_data[15:12];
        3'd4: cur_dig = disp_data[19:16];
        3'd5: cur_dig = disp_data[23:20];
        3'd6: cur_dig = disp_data[27:24];
        3'd7: cur_dig = disp_data[31:28];
    endcase
end

seg7dec dec(.din(cur_dig), .seg(seg));

// 8桁の桁選択（アクティブLOW）
reg [7:0] sel_reg;
always @(*) begin
    case (dig_sel)
```

```
        3'd0: sel_reg = 8'b11111110;
        3'd1: sel_reg = 8'b11111101;
        3'd2: sel_reg = 8'b11111011;
        3'd3: sel_reg = 8'b11110111;
        3'd4: sel_reg = 8'b11101111;
        3'd5: sel_reg = 8'b11011111;
        3'd6: sel_reg = 8'b10111111;
        3'd7: sel_reg = 8'b01111111;
    endcase
end
assign sel = sel_reg;

endmodule
```

Listing 6　simple_top.v

6　テストプログラム

6.1　基本テスト: 加算と出力

最も単純なテストとして、即値ロード・加算・出力・停止を行うプログラムを使用した。

表 7　基本テストプログラム

アドレス	機械語	ニーモニック	動作	期待値
0	0x8003	LI r0, 3	r[0] = 3	r0 = 3
1	0x8105	LI r1, 5	r[1] = 5	r1 = 5
2	0xC800	ADD r0, r1	r[0] = r[0] + r[1]	r0 = 8
3	0xC0D0	OUT r0	output = r[0]	OUT = 8
4	0xC0F0	HLT	停止	halted

6.2　応用テスト: 遅延付きカウンタ

実機デモ用として、遅延ループ付きのインクリメントカウンタを作成した。内側ループ（65,536 回 × 2 命令）× 外側ループ（10 回）で約 0.3 秒の遅延を実現し、カウンタ値を OUT 命令で 7 セグメント LED に出力する。

6.2.1　遅延時間の計算

1 命令あたり 5 クロックサイクル、クロック周波数 20MHz で計算する。

- 内側ループ: $65{,}536 \times 2 \times 5 = 655{,}360$ サイクル
- 外側ループ: $10 \times (655{,}360 + \alpha) \approx 6{,}600{,}000$ サイクル
- 所要時間: $6{,}600{,}000 \div 20{,}000{,}000 \approx 0.33$ 秒/カウント

約 3 回/秒のペースでカウンタが更新され、7 セグメント LED で目視確認可能な速度となる。

表 8　遅延付きカウンタプログラム

アドレス	機械語	ニーモニック	動作
0	0x8000	LI r0, 0	カウンタ初期化
1	0x8101	LI r1, 1	インクリメント値
2	0x840A	LI r4, 10	外側ループ回数
3	0xC800	ADD r0, r1	r0++
4	0xC0D0	OUT r0	出力
5	0x8300	LI r3, 0	外側カウンタ初期化
6	0x8200	LI r2, 0	内側カウンタ初期化
7	0xCA00	ADD r2, r1	r2++
8	0xBBFE	BNE -2	r2≠0 なら addr 7 へ
9	0xCB00	ADD r3, r1	r3++
10	0xE350	CMP r3, r4	r3 と 10 を比較
11	0xB9FA	BLT -6	r3 < 10 なら addr 6 へ
12	0xA0F6	B -10	addr 3 へ戻る

7　動作検証

7.1　テストベンチ

```
// simple_tb.v - SIMPLE 拡張版テストベンチ (担当B更新)
// 4サイクルCPU + BAL/BR/ADDI + I/Oポート拡張の検証
//
// テストモード (TEST_MODE パラメータで切替):
//   0: ISA全命令テスト (test_all.hex) - 期待出力: 0008, 0003, 000A, 0014, FFFF
//   1: バブルソートテスト (sort.hex) - 期待出力: 0000~000F (昇順)

`timescale 1ns / 1ps

module simple_tb;

`ifndef TEST_MODE
`define TEST_MODE 0
`endif
`ifndef INIT_FILE
`define INIT_FILE "test_all.hex"
`endif

parameter TEST_MODE = `TEST_MODE;
parameter INIT_FILE = `INIT_FILE;

reg          clk, rst_n, exec;
wire [15:0] mem_addr, mem_rdata, mem_wdata;
wire         mem_we;
wire [15:0] out_data;
```

```
wire        out_we;
wire [3:0]  io_port;
wire        io_read, io_write;
wire [15:0] debug_pc, debug_r0, debug_r1, debug_r2, debug_r3;
wire        halted;

simple_cpu cpu(
    .clk(clk), .rst_n(rst_n), .exec(exec),
    .mem_addr(mem_addr), .mem_rdata(mem_rdata),
    .mem_wdata(mem_wdata), .mem_we(mem_we),
    .in_data(16'h00FF),
    .out_data(out_data), .out_we(out_we),
    .io_port(io_port), .io_read(io_read), .io_write(io_write),
    .debug_pc(debug_pc),
    .debug_r0(debug_r0), .debug_r1(debug_r1),
    .debug_r2(debug_r2), .debug_r3(debug_r3),
    .halted(halted)
);

ram #(.INIT_FILE(INIT_FILE)) memory(
    .clock(clk),
    .address(mem_addr[11:0]),
    .q(mem_rdata),
    .data(mem_wdata),
    .wren(mem_we)
);

always #25 clk = ~clk;

// ==== ISA テスト用 (TEST_MODE=0) ====
integer out_count;
reg [15:0] expected_isa [0:4];
initial begin
    expected_isa[0] = 16'h0008;
    expected_isa[1] = 16'h0003;
    expected_isa[2] = 16'h000A;
    expected_isa[3] = 16'h0014;
    expected_isa[4] = 16'hFFFF;
end

// ==== ソートテスト用 (TEST_MODE=1) ====
reg [15:0] expected_sort [0:15];
integer i_init;
initial begin
    for (i_init = 0; i_init < 16; i_init = i_init + 1)
        expected_sort[i_init] = i_init;  // 0, 1, 2, ..., 15
end

// ==== 出力監視 ====
```

```
always @(posedge clk) begin
    if (out_we) begin
        if (TEST_MODE == 0) begin
            // ISA テスト
            if (out_count < 5 && out_data == expected_isa[out_count])
                $display("OUT[%0d] = %04h  OK", out_count, out_data);
            else
                $display("OUT[%0d] = %04h  *** MISMATCH (expected %04h) ***",
                        out_count, out_data,
                        (out_count < 5) ? expected_isa[out_count] : 16'hxxxx);
        end else begin
            // ソートテスト
            if (out_count < 16 && out_data == expected_sort[out_count])
                $display("OUT[%0d] = %04h  OK", out_count, out_data);
            else
                $display("OUT[%0d] = %04h  *** MISMATCH (expected %04h) ***",
                        out_count, out_data,
                        (out_count < 16) ? expected_sort[out_count] : 16'hxxxx);
        end
        out_count = out_count + 1;
    end
end

// ==== メインシーケンス ====
initial begin
    clk = 0; rst_n = 1; exec = 0; out_count = 0;
    #50 rst_n = 0; #100 rst_n = 1; #50;

    if (TEST_MODE == 0)
        $display("=== 4-Cycle CPU + ISA Extension Test ===");
    else
        $display("=== Bubble Sort Test ===");

    exec = 1; #50; exec = 0;

    begin : wait_halt
        integer i;
        integer max_cycles;
        max_cycles = (TEST_MODE == 0) ? 2000 : 100000;
        for (i = 0; i < max_cycles; i = i + 1) begin
            @(posedge clk);
            if (halted) begin
                $display("=== CPU Halted (PC=%04h, %0d outputs) ===",
                        cpu.pc, out_count);
                if (TEST_MODE == 0) begin
```

```
                    if (out_count == 5)
                        $display("=== ALL TESTS PASSED ===");
                    else
                        $display("=== SOME TESTS MISSING ===");
                end else begin
                    if (out_count == 16)
                        $display("=== SORT TEST PASSED ===");
                    else
                        $display("=== SORT TEST INCOMPLETE (%0d/16) ===",
                            out_count);
                end
                $finish;
            end
        end
    end
    $display("=== Timeout ===");
    $finish;
end

endmodule
```

Listing 7　simple_tb.v — テストベンチ

テストベンチは CPU とメモリをインスタンス化し、基本テストプログラムを内部メモリに格納して実行する。OUT 命令の出力を`$display` で監視し、HLT による停止を halted 信号で検出する。

7.2　シミュレーション結果

Icarus Verilog 11.0 によるシミュレーション結果を以下に示す。

```
--- Execution Start ---
Time 1225000: OUT = 0008 (8)
--- CPU Halted (PC=0005) ---
```

Listing 8　シミュレーション出力

7.2.1　結果の検証

- **OUT = 0008**: LI r0,3 + LI r1,5 + ADD r0,r1 = 3 + 5 = 8。期待通りの結果。
- **PC = 0005**: HLT 命令（アドレス 4）の実行後、PC は 4+1=5。正しい値。
- **タイミング**: 5 命令 × 5 フェーズ × 50ns/サイクル = 1,250ns。出力タイミング（Time 1225000ps = 1,225ns）はリセット解除後の遅延と exec 検出を考慮すると妥当。

7.3　命令実行トレース

各命令のフェーズごとの動作を表 9 にまとめる。

表 9　命令実行トレース

命令	フェーズ	主要な動作
`LI r0, 3`	p1	IR←0x8003, PC←1
	p3	DR←ALU(0, sext(3), ADD) = 3
	p5	r[0]←DR = 3
`LI r1, 5`	p1	IR←0x8105, PC←2
	p3	DR←ALU(0, sext(5), ADD) = 5
	p5	r[1]←DR = 5
`ADD r0, r1`	p1	IR←0xC800, PC←3
	p2	AR←r[1]=5, BR←r[0]=3
	p3	DR←ALU(3, 5, ADD) = 8; S=0, Z=0, C=0, V=0
	p5	r[0]←DR = 8
`OUT r0`	p2	AR←r[0]=8
	p4	output←AR = 8
`HLT`	p5	running←0

8　性能評価

8.1　命令実行サイクル数

SIMPLE/B では全命令が 5 サイクルで実行される。20MHz クロックでは 1 命令あたり 250ns であり、スループットは 4 MIPS (Million Instructions Per Second) となる。

8.2　回路規模の見積もり

- **レジスタファイル**: 8 × 16 = 128 ビットのフリップフロップ
- **ALU**: 16 ビット加算器 × 1、各種論理演算のマルチプレクサ
- **シフタ**: 16 ビットバレルシフタ（4 段マルチプレクサ）
- **制御部**: フェーズカウンタ（3 ビット）、命令デコーダ（組み合わせ回路）
- **内部レジスタ**: PC, IR, AR, BR, DR, MDR（各 16 ビット）+ SZCV（4 ビット）
- **主記憶**: 256 語 × 16 ビット = 4,096 ビット（Distributed RAM）

Quartus Prime でのコンパイル後、Fitter → Summary で具体的な LE 数を確認すること。

8.3　最大動作周波数

クリティカルパスは以下の経路が候補となる:

- p1: PC → メモリアドレス → メモリ読み出し → IR ラッチ
- p3: BR/AR → ALU 演算 → DR ラッチ
- p4: DR → メモリアドレス → メモリ読み出し → MDR ラッチ

非同期リードメモリ（Distributed RAM）を使用しているため、メモリアクセスを含むパスが最も長くなる可能性が高い。Timing Analyzer で Fmax を確認し、20MHz 以上であることを確認すること。

8.4 改良の方向性

SIMPLE/B は最も基本的な設計であり、以下の改良が考えられる。

- **フェーズの並列実行**: p1/p5 の並列実行で 4 サイクル化、さらに p1/p3 と p2/p5 の並列実行で 3 サイクル化が可能。
- **パイプライン化**: 全フェーズを並列実行し、理想的には 1 サイクル/命令（5 倍の性能向上）を目指す。データハザードやコントロールハザードの対策が必要。
- **Block RAM の利用**: 同期リードの Block RAM に切り替えることで、より大きなメモリ空間を使用可能。ただしメモリアクセスのタイミング調整が必要。

9 まとめ

本レポートでは、SIMPLE アーキテクチャに準拠した 16 ビットプロセッサ SIMPLE/B を Verilog HDL で設計・実装し、シミュレーションにより動作を検証した。

- 5 フェーズ順次実行方式（IF → RR → EX → MA → WB）により、全命令を統一的に 5 サイクルで実行する構成とした。
- ALU（7 演算 + フラグ生成）、バレルシフタ（4 種シフト）、レジスタファイル（8×16 ビット）を独立モジュールとして設計し、CPU コアから接続する階層構造とした。
- 基本テストプログラム（LI/ADD/OUT/HLT）のシミュレーションにより、即値ロード・加算・出力・停止の正常動作を確認した（OUT = 8）。
- 遅延付きカウンタプログラムにより、LD/ST 以外の主要命令（LI, ADD, CMP, B, BNE, BLT, OUT）の動作を検証可能とした。

今後は実機（MU500-RK）への実装と動作確認、およびパイプライン化等のマイクロアーキテクチャ改良を進める予定である。